

SweetLab — Guía Visual y Conceptual

SweetLab — Guía Visual y Conceptual del Proyecto

- **Proyecto:** Full-Stack Web Development
- **Autora:** Mirel Volcan Calderon (MirelSIG)
- **Entorno:** Angular 18, Node.js, MongoDB Atlas
- **Fecha de Entrega:** 26/05/2026 · **Asesor:** Peñascal F5
- **Repositorio GitHub:** github.com/MirelSIG/SweetLab
- **Enlace de Producción (Demo):** sweetlab-kf27.onrender.com

1. Visión General y Arquitectura

SweetLab nació como un laboratorio educativo diseñado para dominar el flujo completo de una aplicación web moderna. Permite la gestión integral de recetas de cocina conectando un ecosistema desacoplado. El Ecosistema Tecnológico

Frontend Visual: Angular 18 (Componentes, Servicios, Formularios Reactivos, HTTP Client).

Backend / API REST: Node.js + Express (Enrutamiento, Controladores, Middlewares).

Base de Datos: MongoDB Atlas (Modelos Mongoose, persistencia en la nube).

Seguridad: Autenticación por Tokens JWT y Control de Roles (admin / lector).

Mapa de Arquitectura del Sistema

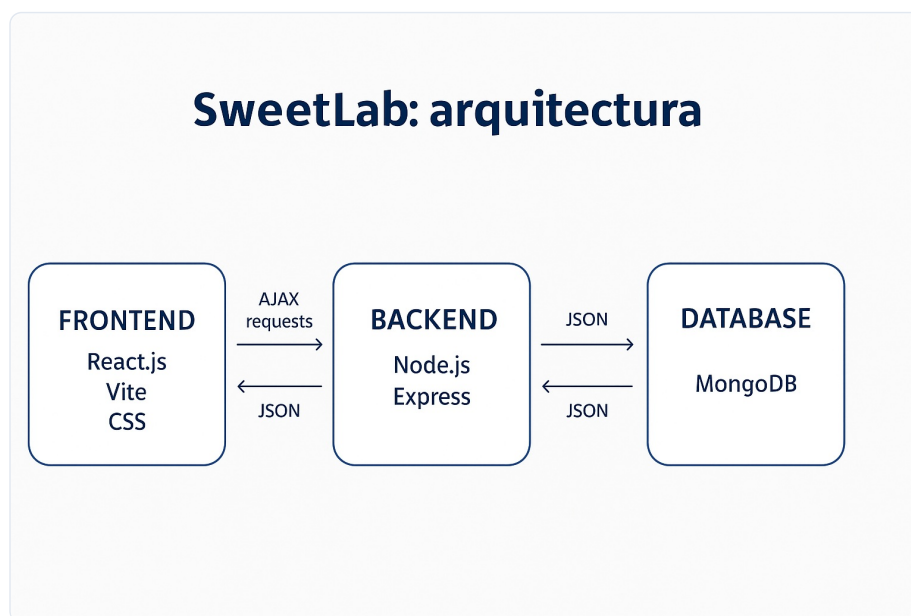


Diagrama de flujo entre el cliente Angular, la API Express y MongoDB Atlas.

2. Estructura del Proyecto (Vista VS Code)

Para comprender el proyecto, es vital entender cómo se organiza el espacio de trabajo. Así se visualiza el árbol de directorios principal en el explorador de Visual Studio Code:

```

SWEETLAB-ROOT
├── backend/
│   ├── config/           # Conexión a Base de Datos (db.js)
│   ├── controllers/      # Lógica de negocio (recipeController.js, authController.js)
│   ├── middlewares/      # Filtros de seguridad (authMiddleware.js)
│   ├── models/           # Esquemas de MongoDB (Recipe.js, User.js)
│   ├── routes/           # Endpoints de la API (recipeRoutes.js, authRoutes.js)
│   └── server.js         # Punto de entrada del servidor Express
└── frontend/
    └── src/
        └── app/
            ├── components/ # Home, Recipe-Form, Login, Navbar
            ├── models/     # Interfaces TypeScript (recipe.model.ts)
            ├── services/   # Consumo de API (recipe.service.ts, auth.service.ts)
            └── assets/     # Imágenes estáticas y respaldos JSON

```

3. El Backend: Diseño de la API REST

El servidor se levanta sobre Node.js y Express, delegando la persistencia a Mongoose. Los bloques de código se estructuran bajo un esquema de color oscuro simulando la sintaxis nativa de VS Code.

Esquema del Modelo de Datos (`models/Recipe.js`)

```

const mongoose = require('mongoose');

const recipeSchema = new mongoose.Schema({
  title: { type: String, required: true },
  ingredients: [{ type: String, required: true }],
  instructions: { type: String, required: true },
  prepTime: { type: Number, required: true },
  image: { type: String, default: 'default-recipe.png' },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Recipe', recipeSchema);

```

Enrutamiento Limpio (`routes/recipeRoutes.js`)

```

const express = require('express');
const router = express.Router();
const recipeController = require('../controllers/recipeController');
const { requireAuth, requireRole } = require('../middlewares/authMiddleware');

// Rutas Públicas (Lectura)
router.get('/', recipeController.getAllRecipes);
router.get('/:id', recipeController.getRecipeById);

// Rutas Protegidas (Escritura - Solo Admin)
router.post('/', requireAuth, requireRole('admin'), recipeController.createRecipe);
router.put('/:id', requireAuth, requireRole('admin'), recipeController.updateRecipe);
router.delete('/:id', requireAuth, requireRole('admin'), recipeController.deleteRecipe);

module.exports = router;

```

4. El Frontend: Cliente Angular 18

El frontend consume los servicios mediante HttpClient, mapeando las respuestas HTTP a interfaces fuertemente tipadas de TypeScript.

Servicio de Datos (`services/recipe.service.ts`)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Recipe } from '../models/recipe.model';

@Injectable({ providedIn: 'root' })
export class RecipeService {
  private apiUrl = 'https://sweetlab-kf27.onrender.com/api/recipes';

  constructor(private http: HttpClient) {}

  getRecipes(): Observable<Recipe[]> {
    return this.http.get<Recipe[]>(this.apiUrl);
  }

  createRecipe(recipe: FormData): Observable<Recipe> {
    return this.http.post<Recipe>(this.apiUrl, recipe);
  }
}
```

5. Interfaz de Usuario y Flujos de Trabajo (Capturas Reales)

A continuación se detallan las pantallas de la aplicación utilizando la interfaz de producción definitiva:

A. Pantalla de Inicio Pública (Home)

Es la vista principal a la que accede cualquier usuario de la plataforma. Muestra el catálogo de recetas disponibles de forma limpia y organizada.



Vista pública del catálogo de recetas de SweetLab con el botón de Acceso Admin.

B. Inicio de Sesión y Autenticación del Administrador

Para realizar tareas de administración se requiere inicio de sesión con token JWT. El administrador introduce su usuario y contraseña para validarse.



Formulario de inicio de sesión.

Verificación de credenciales de administrador.

C. Panel del Administrador Autenticado (Admin Home)

Al iniciar sesión como admin, la interfaz se transforma dinámicamente inyectando los controles de gestión (botones para editar, eliminar y añadir nuevas recetas).



Panel principal de administración con opciones visibles.

Buscador activo y botón dinámico para ocultar acciones.

D. Panel de Gestión y Detalle de Receta

Vista detallada desde la perspectiva del gestor, donde se seleccionan y listan los elementos y se puede acceder directamente a su mantenimiento.



Detalle de receta con opciones integradas para Editar y Eliminar.

E. Creación de Recetas y Subida de Archivos

Formulario reactivo que procesa los datos de texto junto con el archivo binario de la imagen para enviarlo al backend mediante un objeto `FormData`.

Añadir receta

Nombre de tu receta

Ingredientes (ingresa uno por línea o separados por comas)

Pasos (escribe cada paso en una línea)

Tags (separa con comas)

Ej.: #postre, #rápido, #fácil

Tiempo de elaboración (minutos)

Ej.: 45

Nivel de dificultad

Selecciona una opción

Imagen de la receta

assets/mi-receta.jpg o una URL de imagen

Adjuntar imagenGuardar recetaLimpiar

Añadir receta



Media

50 min

Tarta Lunar de Vainilla

Bizcocho suave con crema de vainilla y brillo de azúcar dorado.

#postre #festivo #vainilla

Receta añadida con éxito y mostrada en el catálogo.

Campos vacíos listos para rellenar y cargar la imagen.

F. Modificación de Contenido (Edit Form)

Formulario precargado con el estado actual de la receta seleccionada directamente desde la base de datos de MongoDB Atlas para aplicar actualizaciones parciales o totales.

EDITANDO RECETA

Tarta Lunar de Vainilla

Nombre de la receta

Tarta Lunar de Vainilla

Ingredientes (ingresa uno por línea o separados por comas)

2 tazas de harina
1 taza de azúcar
3 huevos
1 taza de leche
2 cucharaditas de esencia de vainilla

Pasos (escribe cada paso en una línea)

Mezclar los ingredientes secos en un bol grande.
Agregar huevos, leche y mantequilla derretida hasta obtener una masa homogénea.
Hornear a 180 °C durante 35 minutos.
Cubrir con crema de vainilla y terminar con azúcar dorado.

Tags (separa con comas)

postre, festivo, vainilla

Tiempo de elaboración (minutos)

50

Nivel de dificultad

Media

Imagen de la receta

assets/mi-receta.jpg o una URL de imagen

Adjuntar imagenGuardar cambiosCancelar

Campos precargados listos para actualizar en MongoDB Atlas.

6. Resumen de Endpoints de la API

Para propósitos didácticos, esta tabla resume el comportamiento de la API:

#	Método	Endpoint	Acceso	Propósito
01	GET	/api/recipes	Público	Obtener catálogo de recetas completo
02	GET	/api/recipes/:id	Público	Ver detalle de una receta
03	POST	/api/auth/login	Público	Autenticar usuario y devolver Token JWT
04	POST	/api/recipes	Admin	Crear nueva receta (incluye payload de imagen)
05	PUT	/api/recipes/:id	Admin	Editar campos o imagen de receta existente
06	DELETE	/api/recipes/:id	Admin	Eliminar registro permanentemente

-
- **Autora:** Mirel Volcan Calderon (MirelSIG)
 - **Repositorio GitHub:** github.com/MirelSIG/SweetLab
 - **Enlace de Producción:** sweetlab-kf27.onrender.com